

1. Descripción de la aplicación

Objetivo

El sistema de citas médicas busca facilitar el agendamiento de citas para pacientes y la administración de la agenda por parte del personal autorizado.

Funcionalidades principales

- Autenticación de usuarios.
- Gestión de citas (consulta, creación, reprogramación y cancelación).
- Visualización de disponibilidad por fecha/hora.
- Gestión administrativa mediante roles.

Usuarios del sistema

- Pacientes: consultan su historial y agenda.
- Médicos/Recepción: gestionan citas, disponibilidad y datos del sistema.

2. Identificación de activos

Los siguientes activos requieren protección:

- **Datos personales de pacientes**
 - Nombre, documento de identidad, teléfono, correo.
- **Datos de salud asociados a la cita (si existieran)**
 - Motivo de consulta, notas o diagnósticos (según alcance).
- **Credenciales y sesiones**
 - Contraseñas, tokens de sesión, cookies, restablecimiento de contraseña.
- **Base de datos**
 - Tablas de usuarios, médicos, citas y disponibilidad.
- **Servidor y entorno de ejecución**
 - Sistema operativo, configuración, servicios y dependencias.
- **Disponibilidad del servicio**

- La plataforma debe operar sin interrupciones para permitir agendar citas.

3. Diagrama de arquitectura

El diagrama se encuentra en un archivo en el repositorio

4. Riesgos de seguridad

Riesgo 1: Acceso no autorizado / fuga de datos por fallas de autenticación y autorización

- **Descripción:**
Un atacante podría acceder a cuentas o consultar citas/identidad de otros pacientes si existen fallas en autenticación, gestión de sesiones o autorización por roles.
- **Activos afectados:**
 - Datos personales, datos de citas, credenciales, base de datos.
- **Consecuencias:**
 - Violación de privacidad.
 - Exposición de información sensible.
 - Riesgo legal y reputacional.
 - Posible suplantación de identidad.

Riesgo 2: Ataques web (inyección, XSS o CSRF) y manipulación de entradas

- **Descripción:**
Si el backend no valida y sanitiza entradas, un atacante puede ejecutar inyecciones (SQL/NoSQL), XSS o provocar acciones no deseadas mediante CSRF.
- **Activos afectados:**
 - Base de datos, backend API, usuarios, sesiones.
- **Consecuencias:**
 - Alteración de datos (citas).

- Robo de sesiones (XSS).
- Compromiso del sistema (inyección).
- Pérdida de integridad y confianza.

5. Controles de seguridad para mitigar cada riesgo

Controles para el Riesgo 1

- **Autenticación robusta**
 - Hash seguro de contraseñas (bcrypt/argon2).
 - Políticas de contraseña (longitud mínima, bloqueo progresivo).
 - Uso de tokens con caducidad y rotación (JWT con expiración corta o sesiones server-side).
- **Autorización basada en roles (RBAC)**
 - Restringir endpoints y acciones según rol: paciente solo ve/gestiona sus propias citas.
 - Validación en backend (no solo en frontend).
- **Protección de sesiones**
 - Cookies HttpOnly y Secure.
 - CSRF Token si aplica.
 - Logout invalidando sesión/token.
- **Auditoría**
 - Logs de login, fallos y acciones sensibles.

Controles para el Riesgo

- **Validación y sanitización de entradas**
 - Validaciones de tipo, formato y longitud.
 - Sanitización de campos que puedan renderizarse en frontend.
- **Consultas seguras**

- Uso de ORM o consultas parametrizadas.
- Prevención de inyección SQL/NoSQL.
- **Protecciones XSS**
 - Escape/encoding en frontend.
 - Content Security Policy (CSP).
- **Protección CSRF**
 - Tokens CSRF en formularios/acciones sensibles.
 - SameSite en cookies.
- **Gestión de dependencias**
 - Actualizaciones y escaneo de vulnerabilidades (SCA).

6. Relación de controles con el SDLC (tabla)

Fase del SDLC	Riesgo	Control de seguridad	Justificación
Planificación	Acceso no autorizado	Definir requisitos de seguridad (RBAC, protección de datos)	Alinea expectativas desde el inicio y evita “parches” después.
Planificación	Inyección/XSS/CSRF	Requisitos de validación de entradas y hardening	Reduce la superficie de ataque antes del diseño.
Diseño	Acceso no autorizado	Modelo de autorización por roles (RBAC) + diseño de endpoints	Evita que la seguridad dependa solo del frontend.
Diseño	Inyección/XSS/CSRF	Diseño de validaciones + flujo de sesiones (CSRF, cookies)	Define controles de seguridad en la arquitectura.
Desarrollo	Acceso no autorizado	Implementar autenticación segura y control de permisos en backend	Garantiza enforcement real del acceso.

Fase del SDLC	Riesgo	Control de seguridad	Justificación
Desarrollo	Inyección/XSS/CSRF	Parametrización/ORM, sanitización y CSP/encoding	Evita ejecución maliciosa y manipulación de datos.
Pruebas	Acceso no autorizado	Pruebas de autorización (casos por rol) + pruebas de sesión	Verifica que un rol no vea/edite recursos ajenos.
Pruebas	Inyección/XSS/CSRF	SAST/DAST + pruebas de entradas maliciosas	Detecta fallas antes del despliegue.
Despliegue	Ambos riesgos	Configuración segura (HTTPS, headers, secretos en vault) + escaneo final	Reduce exposición y vulnerabilidades del entorno.
Mantenimiento	Ambos riesgos	Monitoreo, parches, reescaneo SCA/SAST y gestión de incidentes	Mantiene la seguridad en el tiempo ante nuevas amenazas.

7. Conclusiones

1. La seguridad debe incorporarse desde la **planificación y el diseño**, ya que corregir fallas en fases tardías es más costoso y riesgoso.
2. La implementación de controles como **RBAC, validación de entradas y protección de sesiones** disminuye significativamente la probabilidad de incidentes.
3. Las pruebas de seguridad (SAST/DAST y pruebas por roles) permiten verificar que los controles funcionan correctamente antes del despliegue.
4. La seguridad continua en mantenimiento (parches, monitoreo y escaneo) es clave porque las amenazas y vulnerabilidades evolucionan constantemente.